

PROJEKTDOKUMENTATION

Einführung eines zentralen IT-Monitorings

AUFTRAGGEBER	AUFTRAGNEHMER	DATUM & VERSION	LAUFZEIT
Müller & Partner GmbH	Grone Umschulungsteam	September 2026 v1.0	3 Projekttag (PoC)

1.

EINLEITUNG & AUSGANGSLAGE

Die **Müller & Partner GmbH** betreibt eine wachsende IT-Infrastruktur mit mehreren Linux- und Windows-Servern für Webdienste, Dateiverwaltung und Netzwerkinfrastruktur (DNS/DHCP). Bisher erfolgte die Systemüberwachung manuell. Dies führte zu verzögerten Störungsmeldungen, unbemerkten Ausfällen von Netzwerkdiensten und potenziellen Datenverlusten durch unentdeckte Festplattenengpässe.

Projektziel: Evaluation, Planung und prototypische Implementierung (Proof of Concept) eines zentralen Open-Source-Monitoringsystems zur proaktiven Störungserkennung vor Beeinträchtigung der Endanwender.

2.

ANFORDERUNGSANALYSE (ABGABEPRODUKT A)

Es wurden 11 verbindliche MUSS-Anforderungen sowie 5 ergänzende KANN-Anforderungen definiert:

ID	Anforderung	Beschreibung	Prio / Status
M01	Zentrales Monitoring	Einrichtung einer zentralen Überwachungsinstanz.	Erfüllt ✓
M02	Multi-System-Setup	Einbindung von mindestens 2 weiteren Testsystemen (Linux/Windows).	Erfüllt ✓
M03	Erreichbarkeit	Erkennung der Erreichbarkeit aller Systeme via ICMP-Ping.	Erfüllt ✓
M04	Systemressourcen	Überwachung von CPU, RAM und Festplattenbelegung.	Erfüllt ✓
M05	Netzwerkdienste	Aktive Überwachung von Anwendungsdiensten (HTTP, SSH, MySQL).	Erfüllt ✓
M06	Schwellwerte	Definition fundierter Warn- und Fehlergrenzen für Metriken.	Erfüllt ✓
M07	Alarmierung	Automatische Erkennung & Visualisierung kritischer Zustände.	Erfüllt ✓
M08	Dashboard	Zentrale Übersicht des Gesamtzustands der Infrastruktur.	Erfüllt ✓
M09	Custom Check	Entwicklung und Integration eines eigenen Messwert-Skripts.	Erfüllt ✓
M10	Störungssimulation	Gezielte Auslösung und Validierung von 3+ Störungsszenarien.	Erfüllt ✓
M11	Abnahmetest	Strukturierter Durchlauf und Protokollierung aller Testfälle.	Erfüllt ✓

Zusätzlich umgesetzt (KANN): E-Mail-Alarmierung (K01), Erweiterte Dashboards (K02) und Detaillierter Custom Check (K04).

3. NUTZWERTANALYSE & SYSTEMAUSWAHL (ABGABEPRODUKT B)

Zur Auswahl des optimalen Monitoring-Systems wurden drei marktführende Open-Source-Lösungen evaluiert:

1. Checkmk Raw Edition

Sehr intuitive Web-GUI (Setup/WATO), automatische Service-Erkennung (Auto-Discovery), umfangreiche Agenten für Linux & Windows, gebrauchsfertige Dashboards.

2. Zabbix

Sehr leistungsfähig und skalierbar, flexible Trigger/Templates. Erfordert jedoch höheren Einarbeitungs- und Einrichtungsaufwand sowie eine externe Datenbank.

3. Nagios Core

Klassischer Standard, extrem ressourcenschonend. Veraltete Benutzeroberfläche, rein dateibasierte Konfiguration, keine integrierten Dashboards.

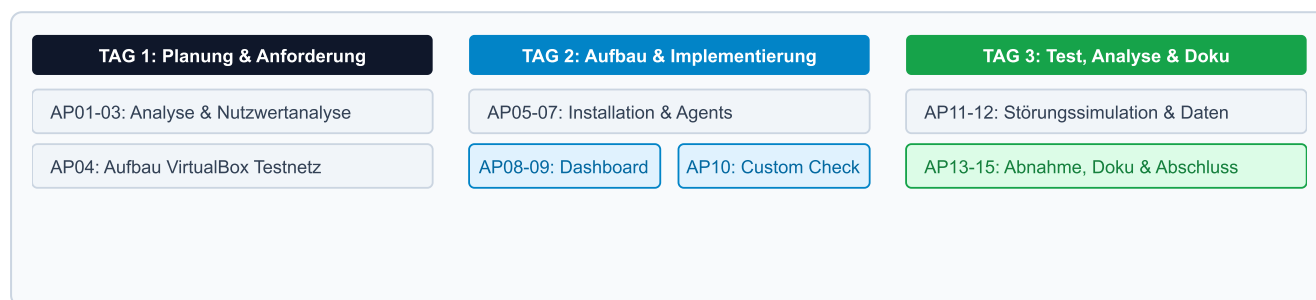
Nutzwertmatrix (Gewichtete Bewertung)

Kriterium	Gewicht	Checkmk Raw	Zabbix	Nagios Core
Kosten / Lizenz	10 %	5 / 0,50	5 / 0,50	5 / 0,50
Bedienbarkeit & GUI	20 %	5 / 1,00	3 / 0,60	2 / 0,40
OS-Unterstützung	10 %	4 / 0,40	5 / 0,50	4 / 0,40
Netzwerkdienste	15 %	4 / 0,60	5 / 0,75	4 / 0,60
Alarmierung	15 %	4 / 0,60	4 / 0,60	3 / 0,45
Visualisierung / Dashboards	15 %	5 / 0,75	4 / 0,60	2 / 0,30
Erweiterbarkeit & Doku	15 %	4 / 0,55	5 / 0,65	4 / 0,65
Gesamtergebnis	100 %	4,40 (Rang 1)	4,20 (Rang 2)	3,30 (Rang 3)

Entscheidung: Die **Checkmk Raw Edition** wird aufgrund der hervorragenden Bedienbarkeit, der integrierten Out-of-the-Box Dashboards und der zeitsparenden Auto-Discovery als Lösung ausgewählt.

4. PROJEKTPLANUNG & ABLAUFSCHEMA (ABGABEPRODUKT C)

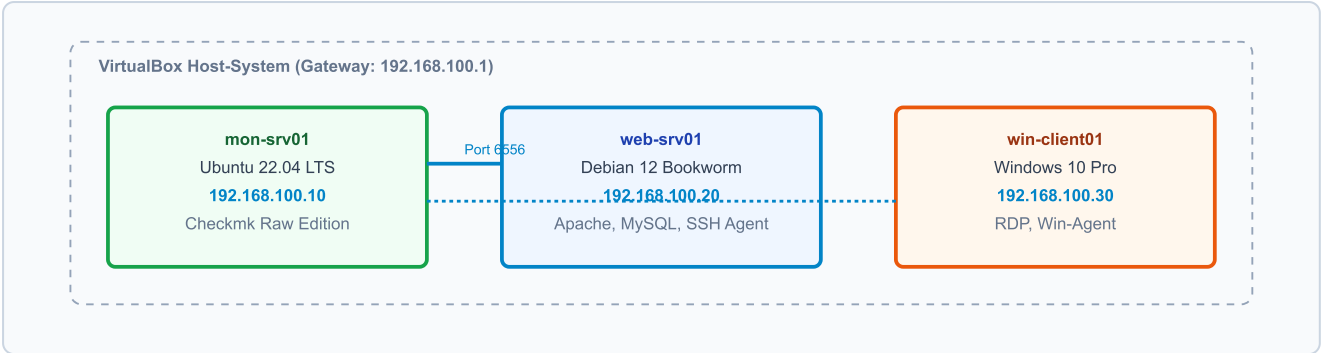
Das Projekt wurde in 15 strukturierte Arbeitspakete gegliedert (Soll-Gesamtdauer: 39,0 h | Ist: 41,5 h). Nachfolgend der zeitliche Ablaufplan:



5.

NETZWERKTOPOLOGIE & SYSTEMÜBERSICHT (ABGABEPRODUKT D)

Die Testumgebung ist als isoliertes virtuelles Netz (192.168.100.0/24) aufgesetzt:



6.

TECHNISCHE DOKUMENTATION (ABGABEPRODUKT E)

6.1

Server-Installation & Agent-Einrichtung

Die Installation von Checkmk erfolgte auf Ubuntu Server 22.04 LTS mittels OMD (Open Monitoring Distribution):

```
sudo apt update && sudo apt install -y ./check-mk-raw-2.2.0p1_0.jammy_amd64.deb
sudo omd create monitoring && sudo omd start monitoring
```

Auf den Zielsystemen wurde der Checkmk Agent eingerichtet und Port 6556/TCP in der Firewall freigeschaltet.

6.2

Definierte Grenzwerte & Schwellwerte (M06)

Messwert / Service	Warnung (WARN)	Kritisch (CRIT)	Begründung
CPU-Auslastung	> 80 % (5 Min)	> 95 % (5 Min)	Puffer für kurze Lastspitzen, verhindert Fehlalarme.
RAM-Nutzung	> 85 %	> 95 %	Vermeidung von intensiver Swap-Nutzung bzw. OOM-Kills.
Festplattenbelegung	> 85 %	> 95 %	Ausreichend Reaktionszeit für Log-Bereinigung/Erweiterung.
HTTP-Antwortzeit	> 2,0 Sekunden	> 5,0 Sekunden	Sicherstellung akzeptabler Webseiten-Ladezeiten.
SSH / Ping	–	Ausfall / Fail	Sofortige Benachrichtigung bei Unreichbarkeit.

7.

EIGENENTWICKLUNG / CUSTOM CHECK (ABGABEPRODUKT F)

Gemäß MUSS-Anforderung M09 wurde ein eigenes Python3-Skript zur Ermittlung aktiver TCP-Netzwerkverbindungen entwickelt und als *Local Check* auf web-srv01 eingebunden:

```
#!/usr/bin/env python3
# Checkmk Local Check: Überwachung aktiver TCP-Verbindungen
import subprocess, sys

def get_tcp_connections():
    try:
        out = subprocess.check_output(["ss", "-ant"], text=True)
        count = sum(1 for line in out.splitlines() if "ESTAB" in line)

        warn, crit = 100, 200
        status = 0
        status_txt = "OK"

        if count >= crit:
            status, status_txt = 2, "CRIT"
        elif count >= warn:
            status, status_txt = 1, "WARN"

        print(f'{status} "Active_TCP_Connections" connections={count};{warn};{crit} '
              f'{status_txt} - {count} aktive TCP Verbindungen')
    except Exception as e:
        print(f'3 "Active_TCP_Connections" - UNKNOWN - Fehler: {e}')

if __name__ == "__main__":
    get_tcp_connections()
```

8.

DASHBOARD-ARCHITEKTUR (ABGABEPRODUKT G)

Das zentrale Monitoring-Dashboard bietet eine hierarchische Übersicht nach dem Prinzip „Vom Überblick zum Detail“:

Taktische Gesamtübersicht

- **Host-Status:** Aggregierte Ampelanzeige (UP/DOWN)
- **Service-Status:** OK, WARN, CRIT, UNKNOWN
- **Event-Log:** Chronologische Alarm-Historie

Performance-Analyse

- **Echtzeit-Graphen:** CPU-, RAM- & Disk-Trends
- **Custom Metrik:** TCP-Verbindungsverlauf
- **Dienst-Status:** HTTP, SSH, MySQL Kennzahlen

9.

STÖRUNGSPROTOKOLL & TESTDURCHFÜHRUNG (ABGABEPRODUKT H)

Zur Verifikation der proaktiven Alarmierung wurden 4 reale Störungsszenarien simuliert:

#	Szenario	Befehl / Aktion	Erwartung	Tatsächliche Erkennung	Reaktionszeit
1	Apache-Ausfall	systemctl stop apache2	HTTP → CRIT	CRIT Connection refused	47 Sek.
2	CPU-Vollast	stress --cpu 4 --timeout 300	CPU → WARN/CRIT	WARN (62s) / CRIT (75s)	62 Sek.
3	Disk voll	dd if=/dev/zero of=/tmp/file	Disk → CRIT	CRIT Belegung > 95%	60 Sek.
4	VM-Ausfall	Power Off win-client01	Host → DOWN	DOWN ICMP Ping Fail	127 Sek.

10.

DATENANALYSE & AUSWERTUNG (ABGABEPRODUKT I)

Analyse Messgröße 1: CPU-Auslastung

Verlauf: Im Normalbetrieb liegt die Auslastung bei 5–15 %. Unter Last (Störung 2) steigt die Kurve vertikal auf 98–100 % an.

Bewertung: Der Grenzwert von 80 % mit einem 5-Minuten-Zeitfenster filtert kurzzeitige Lastspitzen (z.B. Updates) zuverlässig heraus und verhindert Fehlalarme bei maximaler Aussagekraft.

Analyse Messgröße 2: Festplattenbelegung

Verlauf: Normalerweise sehr flacher Anstieg. Bei Befüllung steiler linearer Anstieg im Graphen sichtbar.

Bewertung: Die Warnstufe bei 85 % lässt der IT-Abteilung genügend Reaktionszeit zur Bereinigung oder Vergrößerung des Speichers vor einem Systemstillstand.

Analyse Messgröße 3: HTTP-Verfügbarkeit

Verlauf: Binärer Status (200 OK vs. Connection Refused). Sofortiger Ausschlag im System.

Bewertung: Absolut eindeutig und essenziell für die Überwachung kritischer Unternehmensanwendungen.

11.

ABNAHMETESTPROTOKOLL (ABGABEPRODUKT K)

Test	Anford.	Prüfschritte & Erwartung	Tatsächliches Ergebnis	Status
T01	M01	Aufruf Web-GUI http://192.168.100.10/monitoring	Web-GUI lädt, Login erfolgreich	PASSED
T02	M02	Prüfung eingebundener Hosts	web-srv01 & win-client01 aktiv	PASSED
T03	M03	ICMP-Ping Überwachung auf allen Hosts	PING Status OK auf allen Hosts	PASSED
T04-06	M04	Prüfung Ressourcen-Checks (CPU, RAM, Disk)	Metriken werden korrekt erfasst	PASSED
T07	M05	Prüfung Anwendungsdienste (HTTP/SSH)	Apache Status 200 OK angezeigt	PASSED
T08	M06	Validierung der konfigurierten Schwellwerte	Regeln in Checkmk aktiv & plausibel	PASSED
T09	M07	Generierung eines Alarms bei Störung	Statuswechsel erzeugt Event & E-Mail	PASSED
T10	M08	Prüfung Tactical Overview Dashboard	Gesamtzustand übersichtlich sichtbar	PASSED
T11	M09	Integration des Custom Python Checks	Service Active_TCP_Connections OK	PASSED
T12	M10	Durchführung Störungssimulationen	Alle 4 Störungen korrekt erkannt	PASSED

Das Projekt wurde innerhalb des geplanten Zeitrahmens erfolgreich abgeschlossen. Alle 11 MUSS-Anforderungen wurden zu 100 % erfüllt und durch den strukturierten Abnahmetest nachgewiesen.

Handlungsempfehlung an die Müller & Partner GmbH:

Es wird empfohlen, die **Checkmk Raw Edition** fest in die produktive IT-Infrastruktur zu übernehmen.

Empfohlener Rollout-Fahrplan

1. **Phase 1 (Pilotbetrieb - 1-2 Wochen):** Installation auf dedizierter Hardware/VM und Anbindung der 10 kritischsten Kernserver.
2. **Phase 2 (Vollständiger Rollout - 2-4 Wochen):** Erfassung aller Arbeitsplatzrechner, Netzwerkspeicher (NAS) und Einrichtung von E-Mail-/SMS-Benachrichtigungsketten.
3. **Phase 3 (Erweiterung):** Anbindung von Netzwerkschwitches via SNMP, zentrales Log-Monitoring und Auswertung historischer Daten für die Kapazitätsplanung.

Begriffsglossar

- **Agent:** Überwachungssoftware auf dem Zielsystem.
- **Local Check:** Eigenes Skript zur Metrikerfassung.
- **OMD:** Open Monitoring Distribution Framework.
- **WATO:** Webbasierte Konfigurations-GUI von Checkmk.

Quellenverzeichnis

- Checkmk Doku: docs.checkmk.com
- Zabbix Documentation: zabbix.com/documentation
- Ubuntu Server Guide: ubuntu.com/server/docs
- Linux Manual Pages: man7.org/linux/man-pages